SWEB Interview Guide

What happens where, how to change it — and how to lock it

Theory for the live-coding part of the Abgabegespräche A0, A1, A2

Companion to `preparation/2-ag-interviews/`: the examples (`exNN`), the tasks (`a0/`, `a1/`, `a2/`) and their solutions refer to the sections here, and the sections point back to them. Every claim about SWEB in this guide was checked against your team's code (`osw26e3`, the commit in `team/BASE`) and, where it matters, by running it.

A self-study companion for Operating Systems at TU Graz,
winter semester 2026/27 — not official course material

Contents

1 The interview **3**

1.1 What happens . . . 3
1.2 A strategy for the 30 minutes . . . 3
1.3 How to use this course . . . 4

2 The map: what happens where **5**

2.1 The source tree — the parts you touch . . . 6
2.2 When X happens — the call chains . . . 7
2.3 The two paths you will change most . . . 8
2.4 Who owns what . . . 8
2.5 The address space . . . 9
2.6 Registers: user, kernel, current . . . 9

3 Recipes **10**

3.1 Add a syscall (ex01) . . . 10
3.2 User pointers . . . 11
3.3 Physical pages and mappings (ex02, ex04) . . . 11
3.4 A new page fault case (ex03) . . . 12
3.5 Changing the user registers (ex05) . . . 12
3.6 Threads (ex06, A0-08) . . . 13
3.7 Waiting for something . . . 13
3.8 From another context into a process (ex07) . . . 13
3.9 Disk (ex08) . . . 14
3.10 Kernel-wide objects . . . 14
3.11 Testing traps . . . 14
3.12 Debugging . . . 15

4 Locking — where the points are lost **16**

4.1 Why you need locks on a machine with one CPU . . . 16
4.2 SWEB's locking toolbox . . . 17
4.3 The locks that already exist . . . 17
4.4 The rules . . . 17
4.5 Wrong and right . . . 18
4.6 How to explain your locking . . . 19

5 A0: syscalls, page faults, mappings, registers, threads **21**
5.1 What the hints say . . . 21
5.2 What you must know without looking it up . . . 21
5.3 The mechanics of `pthread_create` in one picture . . . 22
5.4 Tutorial 1: a string from user space . . . 22
5.5 Tutorial 2: one page at two addresses . . . 23
5.6 Variants to expect . . . 23

6 A1: threads, scheduling, processes **25**
6.1 The thread bookkeeping every A1 task needs . . . 25
6.2 Family 1: creating and waiting (join, multiple, join_any, barriers) . . . 26
6.3 Family 2: the scheduler (flagging, invoke, switch, runtime) . . . 26
6.4 Family 3: ending threads (exit, cancel) . . . 26
6.5 Family 4: registers and stacks (snapshot/revive, upcalls, 15 arguments) . . . 26
6.6 Family 5: memory and processes (vtop/ptov, shm, sbrk, page fault statistics) . . . 27
6.7 Not in the course, and how you would do it . . . 27

7 A2: swapping, copy-on-write, page-table bits **28**
7.1 A page-table entry, three meanings . . . 28
7.2 Swap out and in — the order is the whole task . . . 29
7.3 The swap device . . . 29
7.4 Copy-on-write . . . 29
7.5 Moving pages behind a process's back . . . 29
7.6 Sketches for the tasks not in the course . . . 30

8 Your team's code **31**

Chapter 1

The interview

1.1 What happens

In every Abgabegespräch each group member gets a small task, comparable in difficulty to the assignment tasks, and implements it live in the group's SWEB while the tutor watches (official rules page). You also explain code — usually a snippet of your own implementation, often while you implement. If you are slow or cannot explain or implement, you get an *individual* deduction on the assignment's points. The tutor stops you as soon as it is clear that you deserve the points: you do not have to finish a polished solution, you have to show that you *know where things happen and how to change them correctly.*

Students' reports (see `a0/README.md`) agree on three things: the tasks are about **syscalls**, the **page fault handler** and **page mappings**, threads and **registers**; knowing your way around the code matters more than any single algorithm; and the most common reason for deductions is **locking** — missing or wrong (chapter 4). And not every task is a syscall: in our own A0 (14 Oct 2026) the task was `atexit` — pure libc, no kernel change at all.

1.2 A strategy for the 30 minutes

1. **Restate the task** in one sentence and ask about the edge cases you see ("what should happen if the pointer is invalid?"). Tutors expect questions.
2. **Classify it** (`TRIAGE.md`): **U** user space only (a libc stub with a man page, e.g. `atexit`, `malloc`) — **S** syscall + kernel — **E** kernel event (page fault, timer, scheduler, key, boot, exit) — **A** assembly / registers — **K** the kernel calls a user function. Ask: *does it need anything only the kernel has?*
3. **Say where it happens.** "A segfault ends up in `PageFaultHandler::handlePageFault`, and for addresses without a segment in `Loader::loadPage` — I'll hook in there." Chapter 2 is the map for this step.
4. **List the pieces** before typing: syscall (6 places), per-process state (where?), page-fault branch, libc wrapper, test. Chapter 3 has the checklists.
5. **Write the minimal correct version.** No refactoring of existing code, no generality nobody asked for.
6. **Locking pass.** For every piece of shared state you touched: which lock, where taken, where released, could anything run in between? Say it out loud.
7. **Test** with a tiny program in `userspace/tests/` — and show the error path.

Talking while coding

Practise narrating: what you look for, why this file, why this lock. It makes the tutor see your understanding even when you have to look something up, and it keeps you from silently typing yourself into a corner. The tasks' "questions a tutor might ask" are there for this.

1.3 How to use this course

Examples (`examples/`) — one mechanism each, complete and tested; read the patch, apply it, run it. They are your reference when a task needs "that thing with the ident mapping".

Tasks (`a0/`, `a1/`, `a2/`) — task cards in the tutors' format, a test program, hints behind fold-outs, questions. Time-box them.

Solutions (`solutions/`) — patch plus `NOTES.md`. Read the *Locking* and *Pitfalls* sections even when your version works.

Tools — you work in a separate training copy of SWEB, `repos/sweb-ag`, one git branch per task (created once with `task.sh setup`). `tools/task.sh` does the bookkeeping: `task.sh start a0-01` (fresh branch from the right base, tests copied in), `task.sh test` (build + run the tests without a window, with leak check), `task.sh run` / `kvm` / `debug` + `gdb` (exactly like `sweb-practise`), `task.sh reset a0-01` or `reset a1` (start over — your attempt is kept in a `backup/` branch), `task.sh solution a0-01` and `task.sh diff a0-01`.

Chapter 2

The map: what happens where

This chapter answers “which file do I have to touch?”. Read it once completely, then use the tables as a lookup during practice.

2.1 The source tree — the parts you touch

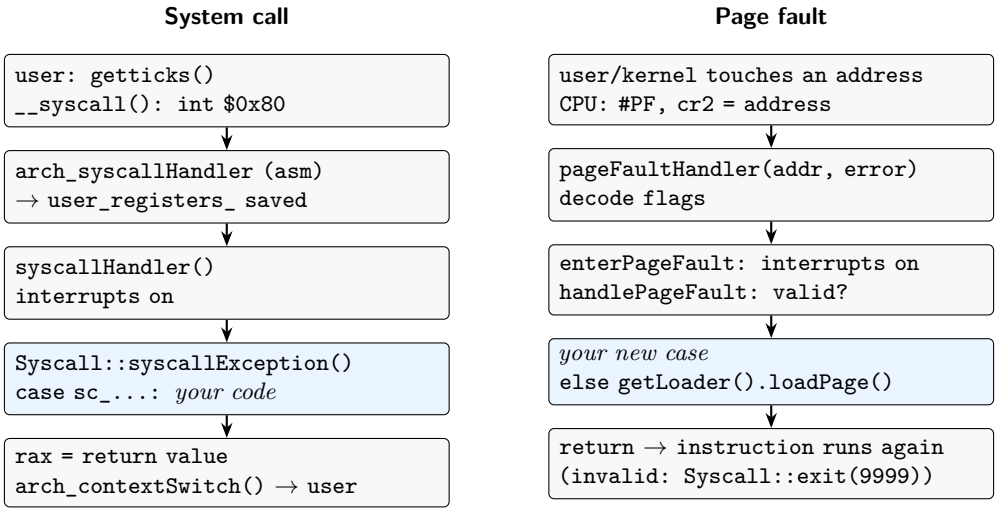
Path	What lives there
`common/include/kernel/syscall-definitions.h`	syscall numbers (`sc_...`) — included by kernel *and* libc
`common/source/kernel/Syscall.cpp`	`syscallException`: the big switch; all syscall implementations
`common/source/mm/PageFaultHandler.cpp`	`enterPageFault`, `checkPageFaultIsValid`, `handlePageFault`
`common/source/kernel/Loader.cpp`	the ELF binary of a process, `loadPage` (lazy loading), owns the address space `arch_memory_`
`common/source/kernel/UserProcess.cpp`	the process: loader, address space (`getAddressSpace()`), `thread_lock_`, threads; `create`, `createThread`, `fork`, `exitThread`/`exitProcess`
`common/source/kernel/UserThread.cpp`	a user thread: first stack page, user registers, address space (your team's code)
`common/source/kernel/Thread.cpp`	`Thread`: kernel stack, registers, state, `kill()`
`common/source/kernel/Scheduler.cpp`	thread list, `schedule()` (round robin), `yield`, `sleep`/`wake`, cleanup
`common/source/kernel/ProcessRegistry.cpp`	starts the programs of `user_progs.h` (the shell) and `createprocess`
`common/source/kernel/{Mutex,SpinLock,Condition,ScopeLock,Lock}.cpp`	locks, with deadlock and misuse checks
`common/source/kernel/main.cpp`	`startup()`: boot order; create kernel-wide objects here
`common/source/mm/PageManager.cpp`	physical pages: `allocPPN` (zeroed), `freePPN`, free/total counts
`common/source/mm/KernelMemoryManager.cpp`	the kernel heap behind `new`/`delete`
`common/source/console/Console.cpp`	`handleKey`: F-keys (F9–F12 debug output, Shift+Fn terminal switch)
`common/source/fs/BDManager.cpp`, `BDVirtualDevice.cpp`	block devices: `idea0`, `idea1` (`/usr`), `idea2` (swap partition)
`common/include/console/debug.h`	debug flags: `debug(FLAG, ...)` on/off per category
`arch/x86/64/source/InterruptUtils.cpp`	C entry of every interrupt: `syscallHandler`, `pageFaultHandler`, `errorHandler`, timer `irqHandler_0`, keyboard `irqHandler_1`
`arch/x86/64/source/arch_interrupts.S`	the assembly stubs: save registers, call the C handler, `iretq`
`arch/x86/64/source/ArchMemory.cpp`	page tables: `mapPage`, `unmapPage`, `resolveMapping`, `checkAddressValid`, `flushTlb`
`arch/x86/64/include/paging-definitions.h`	the page-table entry bit fields (`PageTableEntry`: present, writeable, user_access, dirty, ignored_*, execution_disabled)
`arch/x86/64/source/ArchThreads.cpp`	`createUserRegisters`, `createKernelRegisters`, `setAddressSpace`, atomics
`arch/x86/64/include/ArchThreads.h`	`struct ArchThreadRegisters` (rip, rsp, rax, rdi, ..., cr3)
`arch/x86/64/include/offsets.h`	`USER_BREAK`, `KERNEL_START`, `IDENT_MAPPING_START`
`arch/x86/64/userspace/syscalls.c`	`__syscall`: `int $0x80`, number in `rax`, args in `rbx, rcx, rdx, rsi, rdi`
`userspace/libc/src/*.c`, `userspace/libc/include/*.h`	the libc: `nonstd.c` (SWEB extras), `pthread.c`, `stdlib.c` (`malloc` is a stub!), ...
`userspace/tests/*.c`	one program per file, `name.c` → `name.sweb` on the disk image

2.2 When X happens — the call chains

Event	Chain (file)
System call	user: libc wrapper (`nonstd.c`) → `__syscall` (`arch/x86/64/userspace/syscalls.c`): `int $0x80` → `arch_syscallHandler` (`arch_interrupts.S`): save user registers into `currentThread->user_registers_` → `syscallHandler` (`InterruptUtils.cpp`): `switch_to_userspace_ = 0`, interrupts **on** → `Syscall::syscallException(rax, rbx, rcx, rdx, rsi, rdi)` (`Syscall.cpp`) → return value into `user_registers_->rax`, `arch_contextSwitch()` loads the user registers.
Page fault (#PF, vector 14)	`arch_pageFaultHandler` (asm: `cr2` = address, error code from the stack) → `pageFaultHandler(address, error)` (`InterruptUtils.cpp`): decodes present / write / user / fetch → `PageFaultHandler::enterPageFault` (kernel registers, interrupts **on**) → `handlePageFault`: `checkPageFaultIsValid`? no: backtrace + `Syscall::exit(9999)`; yes: your team's handler first grows the stack (`isUserStackFault`, under `thread_lock_`) or kills on a guard page, otherwise `process->getLoader().loadPage(address, process->getAddressSpace())` (`Loader.cpp`): no segment covers it → `Syscall::exit(666)` → back: the faulting instruction is executed again.
CPU exception (div by 0, #GP, ...)	`errorHandler` (`InterruptUtils.cpp`): prints the error and registers; user thread → `Syscall::exit(888)`, kernel thread → `kill()`.
Timer tick (IRQ 0)	`irqHandler_0`: `Scheduler::incTicks()`, `Scheduler::schedule()` (picks the next schedulable thread, sets `currentThreadRegisters`), `arch_contextSwitch()` — a thread can lose the CPU **between any two instructions** when interrupts are on.
`yield()`	`Scheduler::yield` → `int $65` → `irqHandler_65`: `schedule()` + context switch.
Key press	IRQ 1 → `KeyboardManager::serviceIRQ` (`arch/x86/common`) queues the key → the **Console kernel thread** (`Console::Run`) takes it: printable → terminal; F-keys → `Console::handleKey`.
Program start	boot: `startup()` (`main.cpp`) adds the `ProcessRegistry` thread → `Run()` mounts `/usr`, `createProcess(path)` for each entry of `user_progs.h` (the shell). Shell runs `foo.sweb` → `createprocess` syscall → `ProcessRegistry::createProcess` → `new UserProcess(path)`: `UserProcess::create(path, ...)`: open file, `new Loader(fd)`, read ELF headers, a new `ArchMemory` → the main thread: a stack slot (`allocateNewStack`, one mapped page), `new UserThread(...)`: `createUserRegisters(rip = ELF entry, rsp = stack top)`, `setAddressSpace` → `Scheduler::addNewThread`. The first schedule loads the user registers: the program starts in `_start` (`nonstd.c`) → `main`.
Kernel thread start	`new MyThread` (`Thread(..., KERNEL_THREAD)`: kernel registers with `rip = threadStartHack`) → `addNewThread` → first schedule: `threadStartHack` → `Run()` → `kill()`.
Thread / process end	`exit()` → `Syscall::exit` → `currentThread->kill()`: state `ToBeDestroyed`, `yield()`, never returns. The **CleanupThread** (`cleanupDeadThreads`) removes it from the list and `delete`s it → `~UserThread` → `process->threadExited()`; the last thread deletes the process → `~UserProcess`: `delete loader_`, `delete arch_memory_` → `~ArchMemory` frees every user page and page table; `ProcessRegistry::processExit`.

Event	Chain (file)
Disk I/O	`BDVirtualDevice::readData/writeData` → `BDRequest` into the driver's queue → the thread yields until the IDE interrupt (IRQ 14/15) marks it done.
Kernel `new`	`KernelMemoryManager` (own lock), pages come from the `PageManager`.

2.3 The two paths you will change most



Both run **in the context of the thread that caused them**: `currentThread` is that thread, its page tables are active, `currentThread->user_registers_` holds its user state, and interrupts are enabled — you may take Mutexes, allocate, sleep, do disk I/O. That is not true in the timer interrupt, in `schedule()`, or in the keyboard IRQ.

2.4 Who owns what

Object	Owns / knows
`Thread`	kernel stack (`kernel_stack_`, 8 KiB), `kernel_registers_`, `user_registers_` (user threads), `switch_to_userspace_`, state (`Running/Sleeping/ToBeDestroyed`), its process (`getProcess()`, `getLoader()`), terminal
`UserThread`	a user thread (`: public Thread`): user registers set up in its constructor; its stack slot (`getUserStackTopVpn`, ...)
`UserProcess`	the process (no thread): owns the `Loader`, **the address space** (`getAddressSpace()`), `thread_lock_`, the thread list (`UserThreadData`), working dir, file descriptor. Base SWEB: *is* the one thread of a process (`: public Thread`)
`Loader`	the binary (`fd_`, ELF header, `phdrs_`), `program_binary_lock_`, and `arch_memory_` — **the address space**
`ArchMemory`	the PML4 of one address space (`page_map_level_4_`) and everything below it
`PageManager`	which physical pages are free (bitmap), SpinLock
`Scheduler`	the list of all threads; ticks
`ProcessRegistry`	the number of running processes; starting programs

Per-process state belongs into `UserProcess` (`currentThread->getProcess()`), per-thread state of user threads into `UserThread`. (chapter 8).

2.5 The address space

Address	What
`0x0 - 0xFFF`	never mapped: null pointer accesses fault
`0x400000`, `0x8000000...`	the program: `.text` (R E) at `0x8000000`, `.rodata` (R), `.data`/`.bss` (RW) — `readelf -lW prog.sweb`
`0x5000 0000 0000` etc.	free: this course's tasks put regions here (alloc region, heap at `0x6000...`, scratch `0x7000...`)
`USER_BREAK - 4096`	the main thread's stack page (only this one is mapped at start)
`USER_BREAK` = `0x8000 0000 0000`	end of user space (lower canonical half)
`0x8000...` - `0xFFFF 7FFF...`	non-canonical: any access is a #GP
`KERNEL_START` = `0xFFFF 8000 0000 0000`	kernel half: identical in every PML4
`IDENT_MAPPING_START` = `0xFFFF F000 0000 0000`	**ident mapping**: all physical memory, `getIdentAddressOfPPN(ppn)`
`0xFFFF FFFF 8000 0000`	the kernel image (code, data, the threads' kernel stacks)

In SWEB

Because the kernel half is the same in every address space and the kernel runs with the current process's `cr3`, a syscall can read and write that process's user memory directly through the user pointer. Through the *ident mapping* the kernel can read and write *any* physical page, including pages of other processes and pages not mapped anywhere — that is how page contents are prepared (A0-04), copied (fork, A1), and written to disk (swap, A2).

2.6 Registers: user, kernel, current

`struct ArchThreadRegisters` (`arch/x86/64/include/ArchThreads.h`) holds a full register set: `rip, cs, rflags, rax, rcx, rdx, rbx, rsp, rbp, rsi, rdi, r8-r15, ds, es, fs, gs, ss, rsp0, cr3, fpu`. A user thread has two sets:

`user_registers_` the user-mode state. Saved on every entry into the kernel from user mode (syscall, page fault, interrupt). **While you are in a syscall or a user page fault, it is exactly the state the user thread will continue with** — change it and the thread continues differently (ex05, A0-06, A0-07).

`kernel_registers_` the kernel-mode state, saved when the thread is interrupted *in* the kernel.

`currentThreadRegisters` points to the set that the next `arch_contextSwitch()` loads; `switch_to_userspace_` says which one it is. The syscall and page-fault entries set it to the kernel set while the kernel works, and back to the user set before returning.

Calling convention (System V AMD64): arguments in `rdi, rsi, rdx, rcx, r8, r9`, result in `rax`; at function entry `rsp % 16 == 8`; the 128 bytes below `rsp` are the **red zone** of the running function. SWEB's syscall ABI is different: number in `rax`, arguments in `rbx, rcx, rdx, rsi, rdi`.

Chapter 3

Recipes

Checklists and code templates for every kind of change the tasks need. Each recipe names the example that shows it working.

3.1 Add a syscall (ex01)

	File	Add
1	`common/include/kernel/syscall-definitions.h`	`#define sc_foo 1502` — a free number in your block 1500–1999 (1500 is your `mkdir`)
2	`common/include/kernel/Syscall.h`	`static size_t foo(size_t a, size_t b);`
3	`common/source/kernel/Syscall.cpp`	`case sc_foo: return_value = foo(arg1, arg2); break;`
4	`common/source/kernel/Syscall.cpp`	the implementation
5	`userspace/libc/include/nonstd.h` (or `pthread.h`, ...)	`extern int foo(int a, void* b);`
6	`userspace/libc/src/nonstd.c`	`return __syscall(sc_foo, a, (size_t) b, 0x00, 0x00, 0x00);`

```
size_t Syscall::foo(size_t a, size_t user_ptr)
{
  if (user_ptr == 0 || user_ptr >= USER_BREAK - sizeof(Thing))  // 1. check first
    return (size_t) -1;
  Thing copy = ...;                // 2. work on kernel data
  *(Thing*) user_ptr = copy;       // 3. one copy to user space, at the end
  return 0;
}
```

Trap

- `return -1U;` is `0xFFFFFFFF` — user space sees 4294967295. Use `(size_t) -1`.
- Forgotten `break;` → falls through into the next case.
- `-Werror`: an unused parameter, a missing include, initializers out of declaration order (`-Werror=reorder`) all stop the build.
- `nonstd.h` does not include `types.h` → `unknown type name 'size_t'`.
- At most 5 arguments. More: pass a pointer to a struct or array (A1 "15-argument syscall").
- A new test `.c` needs cmake to run again (`run_test.sh` does it) — and a second `make`: a parallel build may copy the programs to the disk image before they are built (`loading /usr/shell.sweb failed`).

3.2 User pointers

1. **Check the whole range** below `USER_BREAK`, overflow-safe: `p == 0 || p >= USER_BREAK - size` → reject. (`p + size > USER_BREAK` can wrap around.)
2. **Copy in, then check, then use.** Another thread of the process can change user memory between your check and your use (TOCTOU). Copy into a kernel variable first, validate the copy (A0-09: function pointers).
3. **Write results back at the end**, after all locks are released: a user page that is not mapped yet page-faults — in kernel mode, on a user address. That is legal (it is loaded like a normal fault), but it must not happen while you hold a lock, and if no segment covers the address the process is killed right there (order your side effects: A0-08 "touch the id pointer first").
4. Never hand a user pointer to a driver or keep it for later: copy into kernel memory.
5. **Strings have no known length** — there is no range to check up front. Copy byte by byte into a bounded kernel buffer, check *each* address against `USER_BREAK` before reading it, stop at the NUL or the limit, write the NUL yourself (`strncpy` does not when the source is long). Read user memory directly, not via `checkAddressValid`: that one calls a page "unmapped" that is merely not loaded yet (ex11, A0-11).

3.3 Physical pages and mappings (ex02, ex04)

Call	Does
`PageManager::instance()->allocPPN()`	a free physical page, **zeroed**; asserts when memory is full
`PageManager::instance()->freePPN(ppn)`	gives it back (fills it with `0xFF`: use-after-free is detected)
`ArchMemory::getIdentAddressOfPPN(ppn)`	kernel address of that page — valid in every address space
`arch_memory_.mapPage(vpn, ppn, user)`	maps **writable**; `user = 1`: user may access; `false` if vpn already mapped (caller keeps `ppn`); allocates page tables on the way
`arch_memory_.unmapPage(vpn)`	clears the entry, **frees the physical page**, frees empty page tables; asserts if not mapped; **no TLB flush**
`arch_memory_.resolveMapping(vpn)`	`ArchMemoryMapping m`: `m.pt[m.pti]` is the PTE, `m.page_ppn`, `m.page_size` (`PAGE_SIZE` iff a 4 KiB page is mapped; `m.pt` is `nullptr` if there is no page table)
`arch_memory_.checkAddressValid(addr)`	ident address of the byte `addr` if mapped, else 0 — write user memory without risking a fault
`ArchMemory::flushTlb()`	reload `cr3`: forget cached translations

`vpn` and `ppn` are **page numbers** (`address / PAGE_SIZE`), not addresses. **One frame, one mapping:** `~ArchMemory` frees every present user page at exit. A frame you map a second time (an alias, a shared page) must be unmapped *without* freeing first (`unmapPage(vpn, false)`), or it is freed twice: kernel panic `"Double free PPN"` (ex12, A0-12, A1-10).

```
// map a fresh page (outside the lock: allocate; inside: map)
UserProcess* process = currentThread->getProcess();
size_t ppn = PageManager::instance()->allocPPN();
process->getThreadLock().acquire();      // your team's page-table lock
bool mapped = process->getAddressSpace().mapPage(vpn, ppn, 1);
if (mapped && read_only)
{
  ArchMemoryMapping m = process->getAddressSpace().resolveMapping(vpn);
  m.pt[m.pti].writeable = 0;             // same critical section as mapPage
}
process->getThreadLock().release();
if (!mapped)
```

```
  PageManager::instance()->freePPN(ppn); // someone else mapped it first

// unmap
process->getThreadLock().acquire();
if (process->getAddressSpace().resolveMapping(vpn).page_size)
  process->getAddressSpace().unmapPage(vpn);   // frees the ppn - don't free it again
ArchMemory::flushTlb();
process->getThreadLock().release();
```

TLB rule: after *removing* rights (unmap, read-only, NX, swap out) flush the TLB; after *adding* rights (map a non-present page, make writable) you need not — a stale, weaker entry just causes one more page fault that finds everything in order.

The page-table entry (`PageTableEntry`, `paging-definitions.h`): `present`, `writeable`, `user_access`, `write_through`, `cache_disabled`, `accessed`, `dirty`, `size`, `global`, `ignored_2` (3 bits), `page_ppn` (28 bits), `reserved_1`, `ignored_1` (11 bits), `execution_disabled`. The CPU sets `accessed` on every access and `dirty` on every write. When `present == 0`, the CPU ignores *all* other bits — you may store your own information there (A2: the swap slot). `execution_disabled` (NX) works: SWEB enables `EFER.NXE` at boot.

Walking all page tables (A0-01, A2 "all pages of a process"): copy the loop of `ArchMemory::~ArchMemory()` — four nested loops over the present entries, PML4 index `0 .. 255` only (user half).

3.4 A new page fault case (ex03)

- **Where:** `PageFaultHandler.cpp`, function `handlePageFault`: in the branch where `checkPageFaultIsValid` returned true — as its first case, before your team's stack growth and before the call to `loadPage`. Only there you know the fault is a non-present user address.
- **Flags** available: `address`, `user` (fault in user mode), `present` (page was present: protection violation), `writing`, `fetch` (instruction fetch), `switch_to_us`.
- Faults with `present == 1` never reach the valid branch: `checkPageFaultIsValid` rejects them. A protection case (copy-on-write, A2) needs a branch *before* that check.
- The "kill" exits: 9999 invalid fault (page fault handler), 666 no segment (`Loader::loadPage`), 999 file read failed, 888 CPU exception.
- Returning without fixing the cause → the same fault again, forever.

3.5 Changing the user registers (ex05)

Allowed while handling a syscall or a page fault of that thread (its `user_registers_` are saved and will be loaded on return), or for a thread that is not scheduled (not yet added, or sleeping).

```
ArchThreadRegisters* regs = currentThread->user_registers_;
size_t rsp = ((regs->rsp - 128) & ~0xFULL) - sizeof(size_t); // red zone, align, ret
    ↪ slot
pointer slot = loader->arch_memory_.checkAddressValid(rsp);  // under the page-table
    ↪ lock
if (!slot) { /* no usable stack */ }
*(size_t*) slot = return_address;   // e.g. regs->rip (continue after the syscall) or 0
regs->rsp = rsp;
regs->rip = function;
regs->rdi = first_argument;         // rsi, rdx, rcx, r8, r9 for more
```

The syscall's own return value still goes into `rax` afterwards. Why the red zone matters: `objdump`

`-d prog.sweb` → `<__syscall>` keeps its locals at `-0x10(%rbp) ... -0x38(%rbp)` while `rsp = rbp - 8` — *below* `rsp`.

3.6 Threads (ex06, A0-08)

A user thread = same process (`getAddressSpace()`) + own user stack + own registers + an entry in the scheduler. In your team's code `UserProcess::createThread` does exactly this (the course's base version builds it from scratch, A0-08):

```
class UserThread : public Thread { ... };          // Thread(0, name, USER_THREAD,
    ↪ process)
/* a stack slot + its first page under thread_lock_ (allocateNewStack) */
ArchThreads::createUserRegisters(user_registers_, (void*) entry,
                                 (void*) (stack_top - sizeof(pointer)),
                                 getKernelStackStartPointer());
user_registers_->rdi = ...; user_registers_->rsi = ...;
ArchThreads::setAddressSpace(this, process->getAddressSpace());
// later, when everything is set up:
Scheduler::instance()->addNewThread(thread);
```

A kernel thread: subclass `Thread` with `KERNEL_THREAD`, implement `Run()`, `new` it, `addNewThread`. When `Run()` returns, it is killed and the CleanupThread deletes it — never delete a thread yourself, never put one on the stack.

Ending a thread: `currentThread->kill()` — state `ToBeDestroyed`, never returns. The object lives on until the CleanupThread deletes it: **destructors run in the CleanupThread**, when the thread can never run again — the right place to free its stack. Another thread cannot be killed directly "from outside" safely: set a flag that it checks (A1 `pthread_cancel`).

3.7 Waiting for something

```
// waiter                                    // the one who makes it true
lock.acquire();                              lock.acquire();
while (!condition_is_true)                   condition_is_true = true;
  condition_variable.wait();                 condition_variable.signal();   // or
    ↪ broadcast()
/* use the state, still locked */            lock.release();
lock.release();
```

- `Condition(&mutex, "name")`; `wait()` releases the mutex while sleeping and re-acquires it; SWEB **asserts that you hold no other lock** while waiting.
- `signal()` wakes one waiter, `broadcast()` all (barriers, "all threads done").
- `Scheduler::sleep()` / `wake(thread)` are the low-level primitives the locks are built on — using them directly is easy to get wrong (lost wake-ups); prefer a Condition.
- A `while (!x) yield();` loop is *busy waiting*: acceptable for a quick test, not for a solution the tutor should accept ("join must sleep, not spin").

3.8 From another context into a process (ex07)

Key presses are handled in the Console kernel thread, the timer in an interrupt — neither runs in the target process's context. Set a flag there (`ArchThreads::testSetLock(flag, 1)` is an atomic exchange; `atomic_add`, `atomic_set` exist too) and let the target do the work at a safe point in its own context (top of `syscallException`, page fault handler, or a kernel thread you wake).

3.9 Disk (ex08)

`BDManager::getInstance()->getDeviceByName("idea2")` → `readData(offset, size, kernel_buffer)` / `writeData(...)`: offset and size in bytes, multiples of `getBlockSize()` (512); returns size or -1; **blocks** (yields) until done — interrupts on, no SpinLock held. `run_test.sh` boots with a throw-away copy of the disk (`-snapshot`).

3.10 Kernel-wide objects

SWEB never runs constructors of global or static objects. A global `Mutex lock("x");` is zeroed memory and asserts on first use. Create such objects with `new` in `startup()` (`main.cpp`) before the threads that use them exist, or make them members of a singleton that is created at boot.

3.11 Testing traps

- **The timer runs at 18.2 Hz** (`IRQ0_TIMER_FREQUENCY` is 0, the PIT keeps its default divisor 65536): one tick is 55 ms. A round of `sched_yield` passes the IdleThread, which halts until the next tick — **300 yields take about 16 seconds**. Loops that yield look like hangs. Compute instead, or sleep on a Condition.
- QEMU is fast: a busy loop of millions of iterations finishes within one 55 ms time slice. To get a deterministic order between threads, let them yield a different number of times.
- Lazy loading: a page of `.bss` or `.data` that was never touched has no frame yet (`vtop` → -1, `mprotect` fails, statistics are smaller than expected).
- The disk image keeps every program ever built into it; after many tasks in one build folder `exe2minixfs` asserts (`name != ""`). Delete `SWEB-flat.vmdk` and `SWEB.qcow2` in the build folder (`run_test.sh` and `task.sh` do it automatically).
- Tests that must be *killed* can only show it by the absence of a `[FAIL]` line and the kernel's message — put them last in a test program.
- Runner extras: `-n 5` (repeat: races), a final `exit` (SWEB's leak check at shutdown), `-keep-disk` (several boots on one disk), `'&prog.sweb' '!sleep 1' '!key f3' help` (press a key while a program runs).
- User-space libc is small: no `strcpy`, no `sprintf`, `malloc` is a stub; `exit` is not `noreturn` (GCC may warn about "infinite recursion").

3.12 Debugging

Message	Meaning
`Maybe you are dereferencing a null-pointer`	fault below 4096
`pagefault even though the address is mapped`	write to read-only / user access to a kernel-only page
`No section refers to the given address`	`loadPage`: address not in any ELF segment (exit 666)
`You are accessing a kernel address in user-mode`	user pointer $\geq$ `USER_BREAK`
`Double free PPN` / `Page modified after free`	`freePPN` twice / write after free
`thread is holding unrelated locks while waiting on a condition variable`	`wait()` with another lock held
`name_ is null, maybe lock was not initialised?`	a lock whose constructor never ran (global object!)
`don't use strcpy`	use `strncpy`/`memcpy` in the kernel
`You might be leaking physical memory pages`	at shutdown (`exit` in the shell): pages not freed
`Deadlock: Lock ... held already by currentThread`	you acquire a lock you already hold
`CIRCULAR DEADLOCK when waiting for ...`	lock-order cycle — SWEB prints who holds what and waits for what
`Lock ... with IF=0 ... Now we're dead`	lock (or `new`) with interrupts disabled: interrupt handler, `schedule()`

- `debug(FLAG, "fmt", ...)`: categories in `debug.h`; add your own (`const size_t MYFLAG = Ansi_Cyan | OUTPUT_ENABLED;`). `LOADER`, `PAGEFAULT`, `SYSCALL` are on by default.
- In SWEB: **F9** page bitmap + kernel memory, **F10** locks, **F11** stack traces of all threads, **F12** thread list.
- `run_test.sh -raw` shows the whole kernel log; `-n 10` repeats a run.
- gdb: `make debug && make -j && make qemugdb` in the build folder, then `gdb` in a second terminal (see `common_commands.md`).

Chapter 4

Locking — where the points are lost

Ask students where their deductions came from and the answer is almost always the same: a lock forgotten, a lock in the wrong place, a race the tutor spotted in two seconds. This chapter is the checklist that prevents it. Every solution's `NOTES.md` has a *Locking* section that applies it.

4.1 Why you need locks on a machine with one CPU

SWEB runs on one CPU, and still two threads can be "in the middle of" the same code:

- The **timer interrupt** can switch to another thread between any two instructions — in user mode *and in the kernel*: syscalls and page faults run with interrupts enabled (`syscallHandler` and `enterPageFault` turn them on).
- A thread that **sleeps or yields** inside the kernel (a Mutex, disk I/O, `Condition::wait`) lets others run right there.
- **Page faults** happen in the middle of kernel code that touches user memory.

`counter++` is a load, an add and a store. Two threads, a switch after the load: one increment is lost. `if (slot free) map(slot)`: a switch between the check and the map, and two threads map the same slot. These are not theoretical — the timer fires about every millisecond, and the tests in this course run many threads to make it happen.

The core question

For every piece of data you read or write: *can another thread access it at the same time?* If yes (shared between threads of a process, between processes, or between a thread and a kernel thread), it needs exactly one lock that every access takes.

4.2 SWEB's locking toolbox

Tool	Behaviour	Use for
`Mutex`	waiting threads **sleep**; owner tracked; deadlock and misuse checks	the default: anything that may take long, allocate, page-fault or do I/O
`ScopeLock l(m);`	acquires a `Mutex` in the constructor, releases in the destructor	functions with several `return`s — no forgotten release
`Condition(&m, name)`	`wait()` releases `m` and sleeps; `signal()`/`broadcast()`	waiting until some state becomes true (join, barrier, "all threads done")
`SpinLock`	waiting threads **yield in a loop** (busy)	very short sections without sleeping or I/O (the PageManager uses one)
atomics	`ArchThreads::testSetLock(x, v)` (exchange), `atomic_add`, `atomic_set`	a single flag or counter, nothing else depends on it

What the locks **check** (and assert on) — know these, the asserts are your friends:

- acquiring with interrupts disabled → "`Lock ... with IF=0 ... Now we're dead`" (no locks in interrupt handlers, in `schedule()`, or while scheduling is blocked);
- acquiring a lock you already hold → "`Deadlock: ...`";
- a lock-order cycle between threads → "`CIRCULAR DEADLOCK`" with the chain;
- `Condition::wait` while holding *another* lock → assertion ("holding unrelated locks while waiting");
- a thread destroyed while still holding a lock → assertion in `~Thread`;
- a lock whose constructor never ran (global object) → "`name_ is null`".

4.3 The locks that already exist

Lock	Type	Protects
`PageManager::lock_`	SpinLock	the bitmap of free physical pages (inside `allocPPN`/`freePPN`)
`KernelMemoryManager` lock	SpinLock	the kernel heap (inside `new`/`delete`)
`Loader::program_binary_lock_`	Mutex	reading the binary file, `phdrs_` (in `loadPage`)
`ProcessRegistry::counter_lock_` + `all_processes_killed_`	Mutex + Cond.	number of running processes
`Console` / `Terminal` locks	Mutex	screen and input buffers
Scheduler (`lockScheduling`)	internal	the thread list — not for you
page tables (`ArchMemory`)	Mutex	your team's `UserProcess::thread_lock_` (`getThreadLock()`) is taken by the page fault handler for stack pages — use it for every page-table change
anything you add	—	you decide, and you document it next to the declaration

4.4 The rules

1. **One lock per piece of shared state, documented where it is declared.** `Mutex thread_lock_; // protects threads_, the stack slots, the page tables`. Every access — reads too — takes it. A lock taken in the syscall but not in the page fault handler

protects nothing.

2. **Check and act in one critical section.** "Is the slot free?" + "take it", "below the limit?" + "map", "count reached zero?" + "signal". Releasing the lock between check and act is the race (A0-02, A0-03).
3. **Keep critical sections short, but not shorter than the invariant.** Allocate (`allocPPN`, `new`) before taking the lock, commit under the lock, undo outside if the commit failed (free the unused page).
4. **A fixed lock order.** If you ever hold two locks, always take them in the same order everywhere. This course: page-table lock → PageManager/KMM (inside `mapPage`, `new`). Never the reverse. Better still: do not nest at all (A0-08 releases the thread counter lock before creating the thread, which takes the page-table lock).
5. **Never sleep, yield or do I/O while holding a SpinLock.** With a Mutex it is allowed but blocks everybody else for that time — think about whether it must be.
6. **Never die holding a lock.** `Syscall::exit`, `kill()` and anything that may kill the process (a page fault on a bad user pointer!) only after releasing every lock. A dead owner never releases: every other thread of the process hangs.
7. **Condition variables: state + mutex + while.** The state lives in variables protected by the mutex; change it and `signal` while holding the mutex; the waiter loops `while (!state) wait();`. A signal without a waiter is lost — the state is not.
8. **No other lock while waiting** on a Condition (SWEB asserts it).
9. **Locks must outlive their users.** Do not delete (or let the CleanupThread delete) an object while another thread may still be inside one of its locks. Put the lock where the lifetime is clear (ex06: on the waiter's stack; your team: `threadExited()` as the last action of `~UserThread` — and A1-01 shows what happens when a killed thread is still in a Condition's waiters list).
10. **No locks with interrupts off.** Interrupt handlers, `schedule()`, the keyboard IRQ: set an atomic flag, do the work later in a thread (ex07).
11. **Sleep, don't spin.** `while (!done) yield();` burns the CPU and is a classic remark in interviews. Wait on a Condition.
12. **Removing rights ⇒ TLB flush, inside the critical section**, before anyone can rely on the change (unmap, read-only, swap-out).

4.5 Wrong and right

1. **The unprotected counter.**

```
++running_threads_;                          // WRONG: two creators lose one increment
thread_lock_.acquire(); ++running_threads_; thread_lock_.release();   // right
```

2. **The lock in one path only.**

```
// syscall:          ScopeLock l(process->getThreadLock());
    ↪ getAddressSpace().mapPage(...)
// Loader::loadPage: arch_memory_.mapPage(...)   // no lock: your team's loadPage
    ↪ maps without it
```

   Two threads mapping under the same not-yet-existing page table both allocate one and write it into the same page-directory entry: one table, its mapping and its page are lost.

3. **Check, release, act.**

```
lock.acquire(); bool inside = addr < HEAP_START + heap_size_; lock.release();
if (inside) mapPage(...);            // WRONG: heapLimit() may shrink the heap
    ↪ right here
lock.acquire(); if (addr < HEAP_START + heap_size_) mapPage(...); lock.release();
    ↪ // right
```

4. **`if` instead of `while`.**

```
if (!done) cond.wait();       // WRONG: woken up != condition true (broadcast,
    ↪ other waiter)
while (!done) cond.wait();    // right
```

5. **State changed outside the mutex.**

```
done = true; lock.acquire(); cond.signal(); lock.release();   // WRONG: lost
    ↪ wake-up window
lock.acquire(); done = true; cond.signal(); lock.release();   // right
```

6. **Early return without release** — use `ScopeLock`:

```
lock.acquire(); if (bad) return -1; ...  // WRONG: lock held forever
ScopeLock l(lock); if (bad) return -1;   // right: released on every path
```

7. **Dying with the lock.**

```
lock.acquire(); if (!ok) Syscall::exit(9998); ...   // WRONG
lock.acquire(); bool ok_ = ...; lock.release(); if (!ok_) Syscall::exit(9998);   //
    ↪ right
```

8. **Touching user memory under a lock.** `*(size_t*) user_ptr = x;` may page-fault (and kill) — do it after releasing, or write through `checkAddressValid`'s kernel address under the page-table lock.
9. **Lock order inversion.** Thread A: `a.acquire(); b.acquire();` Thread B: `b.acquire(); a.acquire();` → circular deadlock. Pick one order, write it down.
10. **Lifetime.** A mutex inside a kernel-thread object that the CleanupThread deletes while the waiter is still waking up inside `wait()` on it → use after free. Put it where it outlives every user (ex06).
11. **Work in the wrong context.** Changing a process's page tables from the Console thread's key handler, or taking a Mutex in an interrupt handler → flag + deferred work.
12. **Spinning.** `while (thread_alive) yield();` in `pthread_join` → works, but it is busy waiting; the tutor asks you to make it sleep.

4.6 How to explain your locking

Tutors ask *"why do you need a lock here?"*, *"what happens if a thread switch happens right here?"*, *"can this deadlock?"*. Answer in this pattern:

1. *What* is shared: "the page tables of the process and `heap_size_`."
2. *Who else* touches it: "other threads' page faults, `heapLimit` from another thread, the CleanupThread when a thread dies."
3. *Which* lock and *where*: "`thread_lock_`, around check + map in `mapHeapPage`, and in `heapLimit`."
4. *Why no deadlock*: "I hold only this one; `mapPage` takes the PageManager's lock inside, never the other way round."
5. *Why no lost wake-up / no early death*: "`exit` only after the release."

Checklist after every task

1. List every variable/structure you read or wrote that is not local.
2. For each: which lock? Taken on *every* access path (syscall, page fault, destructor, kernel thread)?
3. Every check-then-act inside one critical section?
4. Every `return`/`exit`/`kill`/user-memory access: no lock held, or `ScopeLock`?
5. Two locks at once anywhere? Same order everywhere?
6. Every wait: `while`, state under the mutex, no other lock held?
7. Every object that contains a lock: who deletes it, and can someone still be inside?
8. Rights removed? TLB flushed?

Chapter 5

A0: syscalls, page faults, mappings, registers, threads

5.1 What the hints say

Reported	Trained by
"add to syscalls and page fault interrupts"	every A0 task; recipes §3.1, §3.4
"allocate a page and map it for the user"	A0-02 allocpage, A0-03 heaplimit, A0-04 magic page; ex02
"register manipulation"	A0-07 upcall, A0-06 segv handler; ex05
"segfault → call a function the user registered"	A0-06
"something with the loader"	A0-05 read-only code; ex09
"syscalls and pages"	A0-01 meminfo (page-table walk)
"the mechanics of `pthread_create`"	A0-08
"`pthread_multi`: one thread runs a list of functions"	A0-09
A0 tutorial 1: a user string, a debug flag, per-thread data	ex11 (as written there), A0-11 (done right)
A0 tutorial 2 (7 Oct): physical page of a variable, map it a second time	ex12 (as written there), A0-12 (done right)

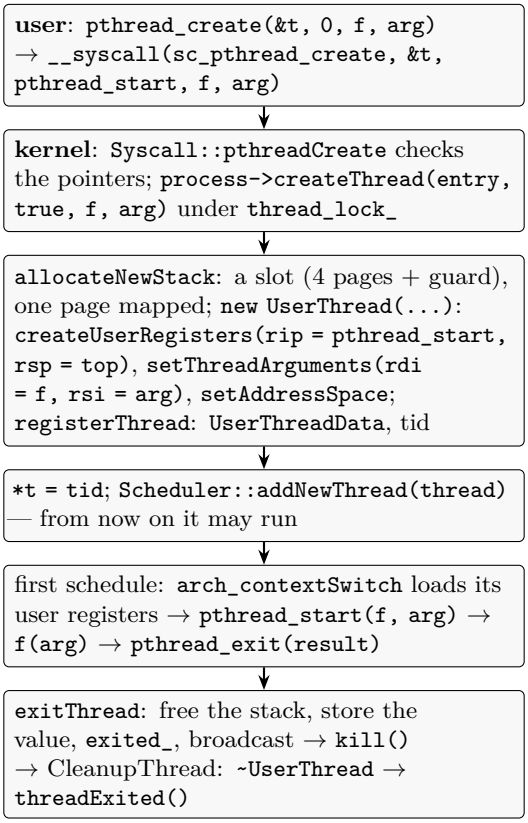
5.2 What you must know without looking it up

1. The six places of a syscall, and the path `int $0x80` → `syscallHandler` → `syscallException` (§3.1).
2. The page fault path and where a new case goes (before `loadPage`); the two places that kill on a segfault (`handlePageFault` 9999, `Loader::loadPage` 666).
3. `allocPPN` → (fill via ident mapping) → `mapPage(vpn, ppn, 1)`; what `unmapPage` frees; when to flush the TLB.
4. `user_registers_` during a syscall/page fault = the state the thread continues with; red zone, alignment, `rdi/rsi/rdx`.
5. A thread = address space (`getAddressSpace()`) + stack + registers + scheduler entry; `createUserRegisters`, `setAddressSpace`, `addNewThread`; destructor in the CleanupThread.
6. Per-process state: where (`UserProcess`) and which lock.
7. The page-table lock: `thread_lock_` (`getThreadLock()`) — and that `loadPage` maps without it.

8. Address arithmetic: `vpn = addr » 12`, `offset = addr & 0xfff`, `addr = (vpn « 12) + offset`; a frame mapped twice is freed twice at exit.

5.3 The mechanics of `pthread_create` in one picture

Your team's `pthread_create` (A0-08 is a study task: explain it):



5.4 Tutorial 1: a string from user space

The first A0 tutorial — reproduced unchanged in ex11 — added a syscall that copies a user string (at most 15 characters) into a `ustl::string s` member of the calling thread, under a per-thread `Mutex s_mutex` (`ScopeLock`), with a debug flag of its own:

```
const size_t TUTORIAL = Ansi_Blue | OUTPUT_ENABLED;      // debug.h: one line per flag

    case sc_tutorial:
      tutorial((char*)arg1);                              // the result is dropped!
      break;
```

What a tutor would ask about it (all fixed in A0-11):

- the `case` has no `return_value =` — every call returns 0, also for `NULL`; and `return -1U` is `0xFFFFFFFF`, not -1;
- only the *first* byte is checked against `USER_BREAK`: a string without NUL right below it makes `strncpy` read `0x800000000000` → General Protection Fault in the kernel, the process is killed (exit 888). A string's end is unknown: copy byte by byte, check each address before reading it;
- `strncpy(temp, string, 15)` leaves `temp` without NUL for long strings — write it yourself;
- `debug(TUTORIAL, "%s", string)` prints the *user* string again: a second, unbounded read after the copy — print the copy;

- the per-thread Mutex: needed only if another thread or kernel path touches the field. Be ready to say which accesses a lock orders — and when none does.

5.5 Tutorial 2: one page at two addresses

The second A0 tutorial (7 Oct) added two syscalls. The tutorial wrote them for upstream SWEB (`((UserProcess*)currentThread`, `process->loader_`); in your team's code `UserProcess` is no `Thread`, so ex12 uses `getProcess()->getAddressSpace()`:

```
size_t Syscall::get_physical_address(size_t virtual_address)
{
  UserProcess* process = currentThread->getProcess();
  ArchMemoryMapping m = process->getAddressSpace().resolveMapping(virtual_address >>
    ↪ 12);
  return m.page_ppn;
}
void Syscall::map_address(size_t virtual_address, size_t ppn)
{
  UserProcess* process = currentThread->getProcess();
  bool mapped = process->getAddressSpace().mapPage(virtual_address >> 12, ppn, 1);
  assert(mapped);
}
```

The user program asks for the frame of a global, maps it at `0x424242424242`'s page, and reaches the global through `(vpn « 12) + (&global & 0xfff)`: the page table translates only the page number, the 12-bit offset is the same in the virtual and the physical address.

Why it panics at exit. `exit` → `kill` → CleanupThread → `~UserThread` → `~UserProcess` → `delete arch_memory_` → `~ArchMemory`, which frees *every* present user page. The global's frame is present twice → `freePPN` twice → `Assertion "Double free PPN"`. The program itself had finished correctly; the backtrace names the CleanupThread.

What a tutor would ask about it (all fixed in A0-12):

- any `ppn` is accepted — a process can map another process's frame or the kernel's code, writable. Only frames already mapped in the caller's own address space (walk the user half like `~ArchMemory`; there is no reverse map);
- no `USER_BREAK` check, and `assert(mapped)`: a user program crashes the kernel by mapping an address that is in use — return `-1`, asserts are for kernel bugs;
- `page_ppn` of an unmapped page is 0, a real frame number — check `m.page_size == PAGE_SIZE`, else `-1` (an untouched page is unmapped: lazy loading);
- no page-table lock — in your team's code `getThreadLock()`;
- the double free: remember the alias vpns, unmap them with `unmapPage(vpn, false)` in `~UserProcess`, right before `delete arch_memory_`;
- `(UserProcess*)currentThread`: wrong in your team's code (compiles, reads garbage) — `currentThread->getProcess()`.

No TLB flush is needed after `mapPage`: the entry was not present before.

5.6 Variants to expect

Practise these as "five-minute variations" of the tasks:

- `mmap`-like: map `n` pages at an address chosen by the kernel / by the user (A0-02 with a length); `munmap` with partial ranges.
- the page fault handler counts faults per process; a syscall returns the count.

- a special address that does something else: returns the time, the tid, a page shared by all processes (careful: `unmapPage` frees the page — shared pages need a reference count).
- tutorial-2 style: the physical page of an address; map a frame of the process a second time; "give me a second address for this variable" — remember: a frame mapped twice is freed twice at exit (§5.5).
- "when a process writes to page X, print a message and continue" (map it on demand, read-only first, then writable on the write fault).
- the segv handler for division by zero (`errorHandler`); with a second argument (read/write); resumable (needs a sigreturn syscall).
- `pthread_create` variants: thread starts with two arguments; a thread that runs a function `n` times; `pthread_multi` with one thread *per* function.

Chapter 6

A1: threads, scheduling, processes

The A1 tasks students reported fall into five families. All of them build on the same few mechanisms: *where* a thread's state lives, *who* frees it *when*, and *how* threads wait for each other. In the course, A1-01 (`pthread_join`) builds that base on top of A0-08; in the interview it is your team's implementation.

6.1 The thread bookkeeping every A1 task needs

Question	Your team's code
Which threads does a process have?	`UserProcess::threads_`: one `UserThreadData` per thread — `tid_`, `exit_value_`, `exited_`, `joined_`, `requested_cancel_`, `thread_` (`nullptr` once it exited)
Which lock?	`thread_lock_` (Mutex, `getThreadLock()`) for all of it — also the stack slots and the page tables; `thread_exited_` (Condition), broadcast in `exitThread`
Where does the exit value survive the thread?	in its `UserThreadData` — the thread object is deleted by the CleanupThread at an unknown time
When is a thread object freed?	`exitThread` frees the stack, stores the value, sets `thread_ = nullptr`, then `kill()`; the CleanupThread deletes it, `~UserThread` calls `threadExited()` (the last one deletes the process)
How do I use another thread safely?	`findThreadDataByTid(tid)` under `thread_lock_`, use `thread_` only while holding it: `exitThread` clears it under the same lock
When is the record freed?	in `joinThread` (after the join), or in `exitProcess`

Locking

- One Mutex for the per-process thread table, one Condition, **broadcast**: different waiters wait for different things (join of thread 3, join of thread 5, exit waiting for all).
- After every `wait()`, re-check *your* condition including identity (the slot may have been joined by someone else and reused).
- Never create a thread object (which allocates and takes the page-table lock) while holding the thread table lock — no nested locks if you can avoid them.
- Never `kill()` a thread while it holds a lock — not even itself.

6.2 Family 1: creating and waiting (join, multiple, join_any, barriers)

Create = shared address space + own stack + own registers + `addNewThread` (A0-08). *All or nothing* (`pthread_multiple`): create every object first, schedule only when all exist; a never-scheduled thread may be deleted directly.

Wait = Mutex + Condition + state, `while` loop. Join waits for FINISHED, `join_any` for "any of these FINISHED, or none alive any more" (then fail instead of waiting forever), barriers for "my round's generation changed" — *not* for a counter that the next round changes again.

6.3 Family 2: the scheduler (flagging, invoke, switch, runtime)

`Scheduler::schedule()` (`common/source/kernel/Scheduler.cpp`) runs in the timer interrupt and in `yield`'s interrupt: **interrupts are off, no lock can be taken**. It walks `threads_`, picks the first `schedulable()` one and rotates the list.

- Data the scheduler reads must be single words written atomically (`ArchThreads::atomic_set`): a "run twice" flag (A1-04), a "run this one next" pointer (A1-05), a per-thread tick counter written in `incTicks` (A1-07).
- A pointer handed to the scheduler may dangle by the time it runs: only use it after finding it in `threads_` (every list member is alive).
- The timer runs at 18.2 Hz (`IRQ0_TIMER_FREQUENCY` 0 → PIT divisor 65536): one tick is 55 ms. A `sched_yield` round passes the IdleThread, which halts until the next tick — so **every yield can cost a whole tick**. Tests (and your solutions) should sleep on a Condition rather than yield in loops.

6.4 Family 3: ending threads (exit, cancel)

- **exit** (POSIX): the whole process ends — your team's `exitProcess` stops all threads *from outside* (`kill()`), which is the same problem as cancel. A thread sleeping in `pthread_join` stays in the waiters list of `thread_exited_` and the kernel panics when the process is deleted — broadcast before killing (A1-01 notes).
- **cancel** (deferred, A1-06): set a flag; the target dies at its next cancellation point (syscall entry, while waiting in join — wake it with a broadcast), never in the middle of a syscall holding locks. A thread that never makes a syscall keeps running.
- The exit value: `PTHREAD_CANCELED` (-1) for a cancelled thread, -1 for one killed by a fault.

6.5 Family 4: registers and stacks (snapshot/revive, upcalls, 15 arguments)

`user_registers_` during a syscall is the state the thread continues with (§3.5). Snapshot/revive (A1-08) restores *registers and the used part of the stack*: registers alone point into frames that have been overwritten since. Copy `ArchThreadRegisters` field by field (its `fpu` pointer and destructor make struct copies dangerous). More than 5 syscall arguments: pass a pointer to an array and copy it in once.

6.6 Family 5: memory and processes (vtop/ptov, shm, sbrk, page fault statistics)

- `resolveMapping` answers virtual → physical; physical → virtual needs a walk of the page tables (no reverse map in SWEB).
- A page in two address spaces needs a **reference count** and an unmap that does not free; process exit (`~ArchMemory` frees every present page!) must drop references first — in `~Loader`, before the member `arch_memory_` dies.
- The page-fault C entry `pageFaultHandler(address, error)` runs with interrupts off: atomics only. `cr2` is read by the assembly stub immediately, because the next fault overwrites it.

6.7 Not in the course, and how you would do it

forkall Fork with every thread: copy the page tables (or COW), then for every thread of the parent create a thread in the child with a copy of its user registers and the same stack slot (`rax` = 0 for the caller). Hold the parent's page-table lock while copying. The design question a tutor will ask: what about a parent thread that is *inside a syscall* right now (e.g. sleeping in `pthread_join`)? Its `user_registers_` hold its state at syscall entry — the child's copy would continue after a syscall it never finished. Options: give the copy the result -1, or stop all threads at the user/kernel boundary first (like cancel's safe points).

special address in the page fault handler A0-04, with a different action: kill only the faulting thread, print all threads' registers (`ArchThreads::printThreadRegisters`).

pthread_switch A directed yield (A1-05) where the caller additionally sleeps until someone switches back to it: `Scheduler::sleep()` after setting the preferred thread — careful with lost wake-ups (the target may switch back before you sleep): use a Condition with a state.

Chapter 7

A2: swapping, copy-on-write, page-table bits

A2 tasks change what a page-table entry means and where a page's content lives. Three ideas carry all of them: **a non-present entry can carry information**, **take the page away before you touch its content**, and **a physical page may have more than one owner**.

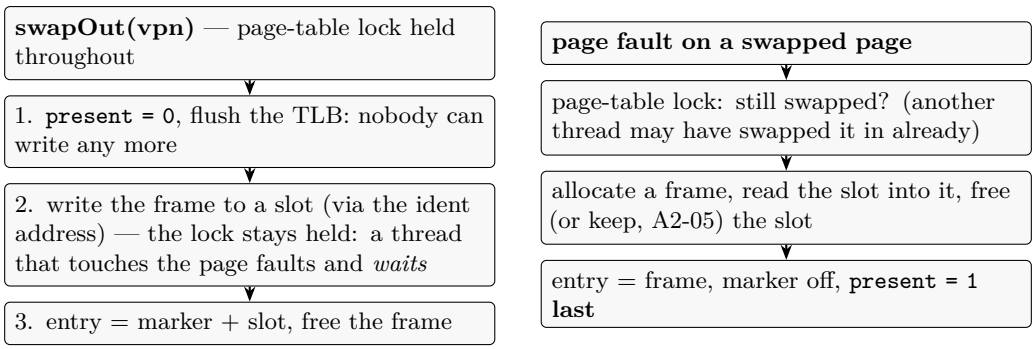
7.1 A page-table entry, three meanings

State	Bits	Meaning
not mapped	all zero	no page here (lazy: `loadPage` on the first access)
present	`present = 1`, `page_ppn` = frame	normal; `writeable`, NX, `dirty`, `accessed` apply
swapped out (A2-01)	`present = 0`, marker in `ignored_2`, `page_ppn` = slot	the CPU ignores all bits of a non-present entry — they are yours
present + clean copy (A2-05)	`present = 1`, `ignored_1` = slot + 1	content also on disk; `dirty` tells whether it is still equal
shared zero page (A2-09)	`present = 1`, `writeable = 0`, `page_ppn` = zero page	the first write copies

Trap

`ArchMemory::unmapPage` frees a page table as soon as no entry in it is *present*. With swapped entries that is wrong: it would throw away the slot numbers. Treat "all entries zero" as empty (A2-01). Likewise `~ArchMemory` frees every *present* page: shared pages (A1-10) and the zero page (A2-09) must be excluded, swap slots (A2-01) freed separately.

7.2 Swap out and in — the order is the whole task



Locking

- **Disk I/O while holding the page-table Mutex is allowed** — and here it is what keeps other threads off the half-written page. A SpinLock would be wrong: the request sleeps.
- One lock order for all of A2: page-table lock → PageManager → swap bookkeeping (`SwapManager`). Your team's swap thread must use the same order.
- Read the `dirty` bit only after `present = 0` + flush — before that a write can still set it.
- Removing rights (present, writable, NX) ⇒ TLB flush in the critical section. Changing the frame of a present mapping (COW, relocation) ⇒ flush.

7.3 The swap device

`idea2` is a 50 MiB partition of type 0x82 that your team's code does not use yet:

```
BDVirtualDevice* swap = BDManager::getInstance()->getDeviceByName("idea2");
```

Slot s = bytes $[s \cdot 4096, (s+1) \cdot 4096)$, written and read through the frame's ident address (`readData`/`writeData`, ex08). Create the bookkeeping object in `startup()` after `BDManager::doDeviceDetection()`; do disk I/O only from threads (the request yields until the IDE interrupt) — `startup()` itself is too early (A2-11 reads its record in `ProcessRegistry::Run()`). `run_test.sh` boots with a throw-away disk; `-keep-disk` keeps one copy over several boots.

7.4 Copy-on-write

A write to a present read-only page faults with `present = 1, writing = 1` — such faults are rejected by `checkPageFaultIsValid`, so the COW branch goes *before* it. CR0.WP is set (`boot.32.C`), so the **kernel's** writes into user memory fault as well: a COW branch that requires `user` breaks every syscall that writes into a fresh buffer. The copy is `allocPPN` + `memcpy` between ident addresses; the zero-page case (A2-09) needs no `memcpy`. Fork's COW (your mandatory part) adds reference counts per frame and two processes' page tables — lock both in a fixed order (e.g. by address) or copy the entries under the parent's lock only.

7.5 Moving pages behind a process's back

Swap-out, relocation (A2-10), defragmentation, "swap out all pages" (F3): the process may run at any moment (preemption!) and touch the page. The pattern is always the same: *present = 0 first, work, present = 1 last*, under the page-table lock; and the page-fault handler, finding a non-present page, must first take that lock and check whether the page is back — *before* treating it as a new

page (`loadPage` would kill the process for a stack page). A thread that works on *other* processes' memory needs a list of address spaces with its own lock, taken *before* any page-table lock, and must keep each process alive while it works on it.

7.6 Sketches for the tasks not in the course

Defragmentation (F7) Walk all swapped entries of all processes (list of address spaces as in A2-10), move slot contents so the used slots are contiguous, update each entry — under each process' page-table lock, one slot at a time; the slot bitmap under its own lock.

Deduplication of arbitrary pages Hash present pages (a kernel thread), compare equal hashes byte by byte, map one frame read-only into both with a reference count, COW on write (A2-09).

mmap a file A region per process; the page fault reads the file part through `VfsSyscall::read` (like `Loader::readFromBinary`); `msync`/`munmap` write dirty pages back.

ASLR Randomise the stack top in `UserProcess` (and the heap start) with `rdtsc`; the code segment needs position-independent binaries.

Checksum check Keep a checksum per swapped page (in a table or in the entry's free bits), verify after swap-in, kill the process (or panic) on mismatch.

Unswappable pages / pages of the own process may not be swapped A flag in the entry's free bits that the swap policy checks; `mlock`-like syscall to set it.

Chapter 8

Your team's code

The interview happens in **your group's repository** — and the whole course runs on it. Since Phase 0 it separates process and thread. State of `osw26e3` `main` on 13 Oct 2026 (commit `40140475`, `team/BASE`) — check again before the interview, the team keeps changing it. Every file with its contents: `ATLAS.md`.

What	Your team's code
process and thread	`class UserProcess` (no thread), `class UserThread : public Thread`; `create`, `createThread`, `fork`, `exec`
the calling process	`currentThread->getProcess()`
page tables	`process->getAddressSpace()` — owned by the `UserProcess`, deleted in `~UserProcess`
page-table lock	`process->getThreadLock()` (`thread_lock_`) — the page fault handler holds it for stack pages, **not** around `loadPage`
the loader	`process->getLoader()` (a `Loader&`): `loadPage(address, getAddressSpace())`
per-process state	members of `UserProcess`
per-thread state	members of `UserThread`: `(UserThread*) currentThread`
a thread by id	`findThreadDataByTid(tid)` under `thread_lock_` → `UserThreadData`
stacks	a slot per thread (`USERSTACK_PAGES` = 4 + guard page), grown in the page fault handler
exit	`exitProcess` kills all threads (POSIX)
syscall numbers taken	1000–1002 `pthread_create/exit/self`, 1500 `mkdir`, 1501–1503 `pthread_join/cancel/testcancel`, 1998/1999, 2000–2002 `fork...`, 2500 `execv`
libc stubs	`malloc/free`, `atexit`, `sem_*`, `pthread_mutex_*`, `pthread_spin_*`, `sleep`, `clock`, `waitpid`, `mmap`, ... (`ATLAS.md`, section 5)

The tutorials were written for upstream SWEB, where the process *is* its thread: read `currentThread->loader_` / `(UserProcess*) currentThread` as `currentThread->getProcess()` — the cast still compiles in your code, but reads garbage.

Found in your team's code

- A thread sleeping in `pthread_join` panics the kernel when another thread ends the process (`exitProcess` kills it, it stays in the waiters list of `thread_exited_`).

- A thread sleeping in `pthread_join` cannot be cancelled (nobody wakes it, the wait loop never checks the flag).
- `loadPage` maps without the page-table lock; `createThread` has no thread limit; `mkdir` starts with `-1U`.

Before the interview: 15 minutes in the team repo

1. Where is the page-table lock? Which locks does `UserProcess` have, what do they protect?
2. Where does `pthread_create` map the stack, set the registers, add the thread?
3. Where are a process's threads listed, and who frees the address space when?
4. Which libc functions are stubs, and which of them need the kernel?
5. `task quiz 10` — then add a trivial syscall in a scratch branch and run it.